

Capacity Planning Overview

Business Central Control Add-in — Architecture, Data Flow & DHTMLX Integration

Extension: DailyOptimizer (Optimizers) — app id range 50600–60700

Page: 50722 "Capacity Planning Overview" | **Controladdin:** DHXCapacityPlanningOverviewAddin

Codeunit: 50604 "DHX Data Handler" — region CPO_

Reference prototype: KLAAS/CPO_v131 (DHTMLXtempv112-app.js / DHTMLXtempv112-data.js)

Folder: src\dhx\capacity_planning_overview\

Document date: 2026-09-03

Table of Contents

1. Overview & Purpose
2. Architecture — BOOT / Wrapper / Controladdin Pattern
 - 2.1 File Inventory
 - 2.2 Single-JS-Component Principle
3. End-to-End Data Flow
4. The JSON Payload Contract (window.DHTMLXPlannerData shape)
5. Section 1 — Stats Header
6. Section 2 — Work Order's Own Day Planning Scheduler
7. Section 3 — Capacity vs Requested Daily Bars
8. Section 4 — Skill → Job/Task → Sequence Tree
9. The Shortage / Coverage Engine (Max-Flow)
10. AL-Side Implementation Details
11. Cross-Work-Order Scoping (2026-09-03 Correction)
12. Performance — Page Background Task Pagination
13. Known Pitfalls, Bugs Found & Fixes Applied
14. Appendix — File Reference

1. Overview & Purpose

The **Capacity Planning Overview** add-in is a Business Central control add-in that gives a single-screen view, for one selected Work Order, of: (a) whether that Work Order's own demand can realistically be covered by the company's resource pool day-by-day, (b) the Work Order's own Day Planning schedule, (c) a daily capacity-vs-request bar chart, and (d) a tree of every *other* Work Order's competing demand in the same visible window.

It is a faithful, BC-native port of a standalone DHTMLX prototype (KLAAS\CP0_v131\DHTMLXtempv112-app.js), with two deliberate substitutions: (1) the prototype's trial/evaluation `@dhx/trial-scheduler` npm package is replaced by this repository's already-licensed `dhtmlxscheduler.js`, and (2) all data is now real Business Central records instead of a static mock JSON file. The prototype's own client-side algorithms (shortage/coverage max-flow engine, tree builder, daily bar aggregator) are ported **near-verbatim** — Business Central's job is reduced to producing one JSON payload shaped exactly like the prototype's own mock `window.DHTMLXPlannerData` object.

Guiding principle (explicit, non-negotiable project requirement): a BC control add-in binds exactly one JS component via one wrapper.js. All four visual sections, the shared header, and the "Days to show" control therefore live inside ONE JS class (`CapacityPlanningOverview`), not four separate `usercontrol` blocks. AL/Business Central is treated purely as a data source and event sink — "consider BC Add-ins as a browser for JS". The only BC-specific surface is the data-communication channel (`SetPlanningData` in, `InvokeExtensibilityMethod` out).

2. Architecture — BOOT / Wrapper / Controladdin Pattern

This follows the same skeleton every other DHTMLX add-in in this repository uses (e.g. `projectschedule` , `request_assignment`):

1. BC loads the controladdin's `StartupScript` , which calls `BOOT()` .
2. `BOOT()` (in `wrapper.js`) finds the host `<div id="controlAddIn">` , normalizes it to fill 100% width/height, renames its id to `cpo-root` , instantiates `new CapacityPlanningOverview('cpo-root')` (stored at `window.__cpo`), and calls `Microsoft.Dynamics.NAV.InvokeExtensibilityMethod("ControlReady", [])` .
3. AL's `ControlReady` trigger (page 50722) fires, which builds the initial JSON payload and calls `CurrPage.DhxCpo.SetPlanningData(json)` .
4. `SetPlanningData` (the controladdin procedure) invokes the JS-side global function of the same name (in `wrapper.js`), which parses the JSON and calls `window.__cpo.applyPlanningData(json)` .
5. `applyPlanningData` is the single entry point that fans out to every section's own `render*` method.

2.1 File Inventory

File	Role
<code>DHXCcapacityPlanningOverviewAddin.ControlAddin.al</code>	Controladdin declaration: script/styleset load order, procedures (AL→JS), events (JS→AL), sizing properties.
<code>page_50722_CapacityPlanningOverview.al</code>	Card page hosting the single <code>usercontrol</code> . No <code>SourceTable</code> . Holds <code>GWorkOrderNo / DaysToShow</code> state, rebuilds and pushes the payload via <code>RefreshData()</code> .
<code>startupScript.js</code>	One line: <code>BOOT()</code> ;
<code>wrapper.js</code>	Thin BOOT/binding shell only — instantiates the JS class, exposes the AL-facing globals (<code>SetPlanningData / SetColors / LoadCapacityLookup</code>).
<code>capacityPlanningOverview.js</code>	The single JS component. All four sections, the shared header, the shortage/coverage engine, scroll-sync, tooltips — everything.
<code>style.css</code>	All layout/heat-map/chip/scrollbar CSS for the four sections and shared header.
<code>codeunit_50604_DHXDataHandler.al</code>	Shared data-handler codeunit (also used by other DHX add-ins). Region <code>CPO_</code> builds the one combined JSON payload.
<code>Pag50662.WorkorderCard.al</code>	Launching page — action <code>CapacityPlanningOverviewAct</code> calls <code>SetWorkOrderNo()</code> then <code>Run()</code> (top-level navigation, not a modal dialog).

Vendor libraries (`dhtmlxscheduler.js/.css`, `suite.js/.css`, `GlobalFunction.js`) are shared, root-level copies reused by every DHX add-in — no per-folder vendor duplication.

2.2 Controladdin Contract

```
controladdin DHXCcapacityPlanningOverviewAddin
{
    RequestedHeight = 1150; MinimumHeight = 420; VerticalShrink/Stretch = true;
    RequestedWidth = 1800; MinimumWidth = 900; HorizontalShrink/Stretch = true;

    Scripts = dhtmlxscheduler.js, suite.js, GlobalFunction.js,
              capacityPlanningOverview.js, // component class BEFORE wrapper.js
              wrapper.js;                 // BOOT() references the class - must load last
    StartupScript = startupScript.js;
    StyleSheets = dhtmlxscheduler.css, suite.css, style.css;

    procedure SetPlanningData(PlanningDataJsonTxt: Text);
    procedure SetColors(ColorsJsonTxt: Text);
    procedure LoadCapacityLookup(CapacityLookupJsonTxt: Text);

    event ControlReady();
    event OnRescheduleWorkOrder(DayShift: Integer; PayloadJsonTxt: Text);
    event OnRequestCapacityLookup(FilterJsonTxt: Text);
    event OnSequenceChipClick(PayloadJsonTxt: Text);
    event OnDaysToShowChanged(NumberOfDays: Integer);
}
```

RequestedHeight = 1150 is sized to the worst case: section 1 (~104px fixed) + section 2 (capped 400px) + section 3 (~180px) + section 4 (capped 420px) + ~40px chrome.
MinimumHeight = 420 with `VerticalShrink = true` lets a small Work Order shrink the control naturally instead of leaving dead space.

3. End-to-End Data Flow

```
Workorder Card (page 50662)
  action CapacityPlanningOverviewAct
    CPO.SetWorkOrderNo(Rec."Work Order No.")
    CPO.Run()                                     ← plain top-level nav, NOT RunModal()
    ↓
page 50722 "Capacity Planning Overview"
  trigger ControlReady()
    EnsureDaysToShow()                           ← DaysToShow default = 30
    RefreshData()
    ↓
  local procedure RefreshData()
    PlanningDataJson :=
      DHXDataHandler.CPO_BuildPlanningDataJson(GWorkOrderNo, DaysToShow)
    CurrPage.DhxCpo.SetPlanningData(PlanningDataJson)
    ↓                                     (controladdin procedure → JS global fn, same name)
wrapper.js
  window.SetPlanningData(json) { window.__cpo.applyPlanningData(JSON.parse(json)); }
  ↓
capacityPlanningOverview.js
  applyPlanningData(json)
    this.db = json; this.dates = [...]; this.woAnchor = 0
    this._baseRequests = this.aggregateRequests()
    this._baselineWithoutWO = this.aggregateOutstandingRequestsWithoutWO()
    renderWoSummaryScheduler(db) → Section 1
    renderWoScheduler(db) → Section 2 (ends by calling renderWorkOrder(), which also
                                     does a first-pass render of Section 3)
    measuredColumnWidth = measureColumnWidth() ← needs a REAL rendered cell from Section 2
    renderCapacityBars(db) → Section 3 (re-rendered with the correct measured width)
    renderCentralTree(db) → Section 4
    bindScrollSync() ← wires the shared horizontal scrollbar to all 4 sections
```

Round-trip back to AL happens on three occasions:

- **"Days to show" changed** (JS-owned input, top bar) → `OnDaysToShowChanged(NumberOfDays)` → AL sets `DaysToShow` and calls `RefreshData()` again (full round-trip, new payload).
- **Section 2 drag / click-to-relocate** → `OnRescheduleWorkOrder(DayShift, payloadJson)` — currently a stub (`Message('Reschedule stub...')`); the real transactional `Validate("Plan Date", ...)` loop is a documented later step.
- **Section 4 chip click** → `OnSequenceChipClick(payloadJson)` — stub, intended to open "Day Plannings" filtered to that specific line.

Section 3's Expand / Exp. to Task / Collapse buttons are deliberately **pure client-side**, no AL round-trip at all — they only toggle Section 4's tree open/closed state (`setTreeOpenState`).

4. The JSON Payload Contract

Built entirely by `codeunit 50604.CPO_BuildPlanningDataJson(WorkOrderNo, NumberOfDays)`, shaped to mirror the reference prototype's own mock `window.DHTMLXPlannerData` object field-for-field. This is the **only** data BC ever pushes into the JS component — every section's render function derives everything else (shortage %, chip layout, bar heights, tree grouping) from this one object client-side.

Field	Type	Meaning / Source
<code>daysToShow</code>	int	Echoed back so the JS-owned input can sync to whatever AL actually used.
<code>startDate</code>	ISO date	<code>Today()</code> — NOT anchored to the Work Order's own Planned Start Date. Also doubles as the implicit workday-offset anchor for Section 2's bars.
<code>endDate</code>	ISO date	<code>startDate + daysToShow - 1</code> .
<code>workdays[]</code>	ISO date[]	Every calendar day in range (weekends included, not skipped) — the reference's own convention.
<code>project.no / .description</code>	obj	The Work Order's parent Job ("Project No.").
<code>workOrder.no / .description / .notEarlierThan / .notLaterThan</code>	obj	The inspected Work Order's own header. <code>description</code> feeds the JS title bar directly — must be the Work Order's <u>own</u> Description, not the parent Job's.
<code>skills[]</code>	array	One entry per distinct Skill demanded (union of the inspected WO's own skills + every other WO's, see §11). Each: <code>code</code> , <code>color</code> , <code>textColor</code> , <code>border</code> , <code>dark</code> , <code>light</code> (colors from codeunit 50609 Visual Default Settings).
<code>resources[]</code>	array	Every resource holding any demanded skill, capped at 15/skill for max-flow performance. Each: <code>name</code> (= Resource No., used as the join key throughout), <code>skills[]</code> .
<code>baseCapacity</code>	number	Flat 8 (hours/resource/day) — matches the reference's own flat model; not a per-resource Res. Capacity Entry lookup.
<code>externalFree[]</code>	number[]	One entry per day in range — summed real "Res. Capacity Entry".Capacity for whichever of the (already-capped) resource pool also has a non-blank Vendor No.
<code>groups[]</code>	array	Section 4's tree source. One entry per distinct Skill demanded by every <u>OTHER</u> Work Order in this window (excludes the inspected WO — see §11). Each has <code>details[]</code> = one entry per distinct Job No./Job Task No. under that skill.
<code>dayPlanningLines[]</code>	array	Real Day Planning rows: the inspected WO's own lines plus every other WO's lines in the same window. Each line carries <code>workOrderNo</code> (normalized — see §12) so client code can tell which WO it belongs to.
<code>workOrderSequences[]</code>	array	Section 2's source. One entry per distinct Skill+Job+Task+SequenceNo combination belonging ONLY to the inspected Work Order, plus a per-(sequence, workday-offset) requested-hours map.

dayPlanningLines[] — per-line shape

```
{
  id, workOrderNo, job, task, description, sequenceLineNo,
  requestDate, requestedStartTime, requestedEndTime, requestedHours, requestedSkill,
  assignedResourceNo, assignedDate, assignedStartTime, assignedEndTime, assignedHours,
  sequenceNo
}
```

Function: `renderWoSummaryScheduler(json)` | **Host div:** `#cpo-wo-summary` | **View name:** `workordersummary`

A 2-row synthetic `Scheduler.getSchedulerInstance()` timeline (NOT a real event calendar — it has no real events, only `cell_template`-rendered HTML per cell). The two rows are “Calculated conclusion” (% coverage + added shortage hours) and “Current position shortage” (hours caused specifically by this Work Order’s own current schedule position).

createTimelineView option	Value	Note
render	'bar'	Standard timeline, not tree.
x_unit / x_step / x_size	day / 1 / dates.length	One column per visible day.
y_unit	2 synthetic sections	{key: 'fit', ...}, {key: 'shortage', ...} — not real DB rows.
dy	CALC_ROW_HEIGHT = 34px	
column_width	COLUMN_WIDTH = 92px	Identical across all 3 Scheduler instances (1/2/4) — DHTMLX auto-stretches columns wider than this literal value unless total content already exceeds container width.
dx	ROW_LABEL_WIDTH = 360px	Row-header width — honored literally regardless of container width.
scale_height	SCALE_HEIGHT = 36px	The shared date-header row sections 1–3 all align under.
cell_template	true	Every cell's class AND value are custom HTML, computed live.

Data source: nothing from the payload directly — every cell calls `this.evaluateW0(dayIndex) / this.currentPositionShortage()[dayIndex]` LIVE at render time (see §9, the max-flow engine), which themselves read `this.db.dayPlanningLines`, `this.db.resources`, `this.db.externalFree`, `this.db.workOrderSequences`. There is no AL-precomputed “statsRows” array (an earlier architecture had one; it was deliberately removed in the 2026-09-02 max-flow pivot).

Function: `renderWoScheduler(json) → renderWorkOrder()` | **Host:** `#cpo-wo-scheduler` (wrapped by `#cpo-wo-scheduler-wrap`, see §12) | **View:** `workorder`

A [real](#) Scheduler timeline with native drag-move enabled. One row per distinct Skill+Job+Task+SequenceNo combination the inspected Work Order itself demands (from `workOrderSequences[]`). Each row's events are positioned via a workday-offset formula (`idxWork(woAnchor, offset)`), not raw calendar "Plan Date" placement — dragging a bar shifts a single client-side `woAnchor` integer and re-renders the whole section instantly (optimistic, pure client-side simulation matching the reference exactly), then fires `OnRescheduleWorkOrder` to AL.

createTimelineView option	Value	Note
render	'bar'	
y_unit	one section per sequence	Built from <code>json.workOrderSequences</code> ; falls back to a single "No Day Planning lines" placeholder row if empty.
dy / event_dy	32 / 24px	<code>ROW_HEIGHT / ROW_HEIGHT - 8</code> .
dx	360px	Same <code>ROW_LABEL_WIDTH</code> as section 1 (shares its date header).
scale_height	0	Deliberately zero — shares section 1's header row instead of drawing its own. DHTMLX still renders an empty <code>.dhx_cal_header / .dhx_scale_bar</code> regardless — force-hidden via CSS (<code>#cpo-wo-scheduler .dhx_cal_header{display:none!important}</code>).
config.drag_move	true	The only section with native drag.

Events (drag / click):

- `onEventChanged` — computes the day-shift from the drop position, updates `woAnchor`, re-renders, then `InvokeExtensibilityMethod('OnRescheduleWorkOrder', [shift, json])`.
- `onClick` — opens the (stubbed) Capacity lookup modal for that event's skill/day.

Data source: `json.workOrderSequences[]` for row structure; per-cell values pull from `workOrderAssignmentState`, which reads `dayPlanningLines[]` filtered to `line.workOrderNo === inspectedWorkOrderNo` (see §11/§12 for why this must be the *normalized* `workOrderNo`, not always the raw table field).

Bug fixed 2026-09-03: this section used to only pick up rows whose raw "Work Order No." table field equalled the inspected WO — but some of a Work Order's own genuine Day Planning Sequences (e.g. added via the Workorder Card's native "New sequence" button) can carry a [blank](#) value in that field. AL now scopes by `Job No. = WorkOrder."Project No."` AND `Job Task No. = WorkOrder."Project Task No."` instead — matching the native "Day Planning Sequence" part on the Workorder Card's own filter semantics.

Section 3 — Capacity vs Requested Daily Bars

Function: `renderCapacityBars(json)` | **Host:** `#cpo-capacity-bars`

This section is **deliberately not a DHTMLX widget at all** — it is plain generated HTML/CSS: for each visible day, a stacked “C” (Capacity: Assigned → Internal free → External free) column and a stacked “R” (Requested: Assigned → Unassigned-per-skill) column, plus Totals/Expand/Exp. to Task/Collapse buttons that control Section 4’s tree open state (pure client-side, no AL round-trip). Column width is **measured**, not the literal `COLUMN_WIDTH` constant — `measureColumnWidth()` reads a real rendered `.dhx_matrix_cell` from Section 2 (or the summary scheduler) after its first paint, since DHTMLX auto-stretches columns whenever total content is narrower than the container.

Data function	Reads	Produces
<code>capParts(dayIndex)</code>	<code>dayPlanningLines[]</code> (ALL work orders, unfiltered), <code>resources[].length × baseCapacity</code> , <code>externalFree[dayIndex]</code>	<code>{assigned, freeInt, freeExt}</code> — the “C” column. Company-wide on purpose: a resource committed to any other Work Order that day is genuinely unavailable.
<code>dailyCapacityRequestData()</code>	<code>capParts()</code> + <code>dayPlanningLines[]</code> filtered to <code>line.workOrderNo === inspectedWO</code>	One row per visible day: <code>assigned, freeInt, freeExt, request, assignedRequest, unassignedBySkill{}</code> , <code>shortage</code> .

Why the “R” (Requested) side IS work-order-filtered but the “C” (Capacity) side is NOT (explicit 2026-09-03 user correction, §11): Section 3 answers “how does this Work Order’s own demand sit against the company’s real, fully company-wide capacity picture” — the capacity side was already correct before the cross-WO change; only the demand side needed an explicit filter once `dayPlanningLines[]` started carrying every other Work Order’s rows too (to feed Section 4).

Function: `renderCentralTree(json)` | **Host:** `#cpo-central-tree` (wrapped by `#cpo-central-tree-wrap`, see §12) | **View:** `centraltree`

The third real Scheduler instance, using `render:'tree'` (the `treeline` render mode — there is **no dhtmlx-gantt library anywhere** in this add-in). Three levels: Skill (folder) → Job/Task “detail” row (folder) → Sequence (leaf). Leaf day-cells can render multiple side-by-side “chip” cells when several real Day Planning lines land on the same day/sequence; parent Skill/Task rows show one aggregated heat-map percentage cell instead.

createTimelineView option	Value	Note
<code>render</code>	<code>'tree'</code>	Only section using tree mode.
<code>y_unit</code>	<code>buildCentralSections()</code> output	Client-built Skill→Task→Sequence hierarchy from <code>groups[]</code> + <code>dayPlanningLines[]</code> .
<code>dy / folder_dy</code>	<code>ROW_HEIGHT = 32px</code> (both)	
<code>section_autoheight</code>	<code>false</code>	Intent: literal row height. In practice DHTMLX still fits total row height to whatever its own <code>init-target</code> element's <code>clientHeight</code> reports – see §12's bug fix.
<code>dx / column width in header</code>	<code>TREE_LABEL_WIDTH = 360px</code> (3-column: Skill/Job/Task)	Wider than sections 1–3's shared 360px row label — here it's a real 3-column grid header, own independent date row (<code>scale_height</code> nonzero, unlike section 2).
<code>columns[0].template</code>	<code>centralTreeLeftColumnHtml(o)</code>	Renders the 3-column Skill Job Task left panel per row (CSS grid <code>176px 92px 92px</code>).

Per-row/cell templates:

- `centraltree_cell_value` — Skill rows call `skillDaySummary()`; Job/Task “detail” rows call `taskDaySummary()` (both render a gradient-filled % heat cell); Sequence leaf rows call `sequenceDayCellHtml()` (renders the actual chip(s)).
- Chips are clickable/hoverable — `attachTreeChipTooltip()` binds one shared mouseover/click handler on the whole host div (event delegation via `.closest('.cpo-tree-chip')`), reusing the same tooltip element Section 2 uses. Click raises `OnSequenceChipClick`.

Bug fixed 2026-09-03 (scope): `groups[]` (and therefore this whole tree) is built AL-side exclusively from every OTHER Work Order's demand in the visible window — the inspected Work Order's own data must never appear here (it already has its own view in Section 2). The exclusion criterion had to be corrected from the raw “Work Order No.” field (unreliable — see §6) to a Job No./Job Task No. match against the inspected Work Order.

Bug fixed 2026-09-03 (V-scrollbar): DHTMLX was auto-shrinking every row's rendered height to fit whatever height its own `init-target` element reported, regardless of `dy / section_autoheight` config — so a tree with far more rows than fit never actually overflowed, and no scrollbar ever appeared. Fixed by splitting one div into two: DHTMLX now initializes into an unconstrained inner div (grows to its true `rowCount × dy` height), wrapped by an outer div that carries the MIN/MAX-clamped height and `overflow-y:auto` — confirmed live: inner content 90,156px tall, clipped to a real, functioning 420px scrollable window.

9. The Shortage / Coverage Engine (Max-Flow)

Section 1’s two rows and Section 3’s implicit “shortage” figure are computed by a near-verbatim port of the reference prototype’s own client-side algorithm — an Edmonds-Karp-style augmenting-path max-flow model (`maxFlowDay`), not a simple “sum capacity vs sum demand” formula.

Function	Purpose
<code>maxFlowDay(dayIndex, demandBySkill)</code>	Given one day’s per-skill demand, computes the maximum achievable coverage across the (skill-scoped, capped) resource pool via flow conservation — achievable coverage is capped by demand , not supply, so a larger resource pool never inflates coverage past 100%.
<code>evaluateWO(dayIndex)</code>	“What if this Work Order’s current schedule position is included?” — returns <code>{coverage, addedShortage}</code> , the source of Section 1’s “Calculated conclusion” row.
<code>currentPositionShortage()</code> / <code>currentPositionSkillShortage()</code>	Before/after delta: <code>maxFlowDay</code> WITHOUT this WO’s demand vs WITH it, at its current <code>woAnchor</code> position — the source of Section 1’s “Current position shortage” row and Section 2’s per-row shortage badges.

Why the algorithm was reversed back in (2026-09-02 decision): an earlier, simplified “sum team capacity minus other-WO-assigned-hours” formula had a real bug — a large company-wide resource pool produced coverage numbers over 5,700%. The max-flow model doesn’t have this failure mode because achievable throughput is fundamentally bounded by demand, never by an oversized supply pool.

Performance caveat: max-flow is expensive (roughly $O(\text{resources}^2)$ per day) and is called from BOTH Section 1’s per-cell templates AND the shortage functions’ own per-day loops, with no memoization in the reference. This port adds per-render-pass caching (`this._evalWOCache` , `this._currentPositionShortageArr` , cleared on every `applyPlanningData()` / `resetAnchorDependentCaches()`) and AL caps the resource pool at **15 per skill** (`CPO_MaxResourcesPerSkill`) — both are deliberate, documented performance trade-offs, not data-completeness bugs.

10. AL-Side Implementation Details

codeunit 50604 “DHX Data Handler” — CPO_ region, key procedures

Procedure	Purpose
CPO_BuildPlanningDataJson(WorkOrderNo, NumberOfDays): Text	The one entry point — assembles the entire payload described in §4, in 3 passes over Day Planning (see below).
CPO_BuildDayPlanningLineObj(DayPlanning, InspectedWorkOrderNo, InspectedJobNo, InspectedJobTaskNo): JsonObject	One <code>dayPlanningLines[]</code> entry. Normalizes <code>workOrderNo</code> to the inspected WO's own No. whenever the line's Job No./Job Task No. match it, regardless of the raw table field (bug fix, §12).
CPO_BuildSkillsArray / CPO_BuildResourcesArray / CPO_BuildExternalFreeArray / CPO_BuildGroupsArray	Build <code>skills[]</code> / <code>resources[]</code> / <code>externalFree[]</code> / <code>groups[]</code> respectively — see §4 for each one's exact scope.
CPO_BuildWorkOrderSequencesArray	Builds <code>workOrderSequences[]</code> from the Pass-1 dedup lists.
CPO_ComputeWorkdayOffset / CPO_IndexOfText / CPO_BuildDateRangeArray / CPO_FormatTimeHHMM	Small shared helpers (offset math, list lookup, date range, time formatting).
CPO_MaxResourcesPerSkill(): Integer	Returns 15 — the max-flow performance cap (§9).

CPO_BuildPlanningDataJson — the 3 passes

```
Pass 1 (DayPlanning, scoped: Job No. = WorkOrder."Project No." AND Job Task No. = WorkOrder."Project Task No.")
    → dayPlanningLines[] (this WO's own rows) + dedup into ActiveSkillList / SeqOrder (workOrderSequences source)

Pass 2 (same recordset, second loop, now that SeqOrder is fully populated)
    → per-(sequence, workday-offset) requested-hours accumulation (SeqOffsetHours)

Pass 3 (OtherDayPlanning, scoped: Skill <> '' AND Plan Date in [StartDate,EndDate];
    in-code skip of rows matching the inspected WO's own Job No./Job Task No.)
    → appended into dayPlanningLines[] + groups[] (every OTHER Work Order's demand)
```

Pass 1 and Pass 3's exclusion both key off **Job No./Job Task No.**, not the raw `"Work Order No."` field — AL cannot express `"NOT(A AND B)"` as a single `SetRange / SetFilter` chain (those only AND conditions together), so Pass 3's exclusion is an in-code `if not (...)` guard inside the loop, not a table filter.

page 50722 “Capacity Planning Overview”

- No `SourceTable` — a bodiless Card page holding exactly one `usercontrol`.
- `Caption = ''`; — blank on purpose (the JS renders its own title bar; a non-blank Caption would duplicate it as a native BC heading).
- `SetWorkOrderNo(pWorkOrderNo)` — filter-setter-before- `Run()`, called by the launching page before opening.
- `DaysToShow` global var, default 30 (`DefaultDaysToShow()`) — the ONLY page-level state; everything else lives in the JSON payload.
- `RefreshData()` — single shared rebuild-and-push routine called by `ControlReady` and `OnDaysToShowChanged`.

Pag50662.WorkorderCard.al — launching action

```
action(CapacityPlanningOverviewAct)
{
    trigger OnAction()
    var CPO: Page "Capacity Planning Overview";
    begin
        CPO.SetWorkOrderNo(Rec."Work Order No.");
        CPO.Run();    // plain top-level navigation - see section 12's UI-chrome fix
    end;
}
```

11. Cross-Work-Order Scoping (2026-09-03 Correction)

Explicit product decision, stated by the user in these exact terms: *"section 3 for DWO0008 but section 4 for all day planning data in days time frame except DWO0008"*, further clarified as scoped "as long as in same time frame of days" and explicitly including "other WO in other job" (i.e. truly company-wide, not scoped to the inspected WO's own parent Job).

Section	Scope	Rationale
Section 1 (stats)	Inspected WO only	"This WO's own position".
Section 2 (own scheduler)	Inspected WO only	Literally IS the inspected WO's own schedule.
Section 3 — "R" (Requested)	Inspected WO only	"This WO's own demand."
Section 3 — "C" (Capacity)	Company-wide (unfiltered)	A resource committed elsewhere that day is genuinely unavailable — already correct before this change.
Section 4 (tree)	Every OTHER Work Order, ANY Job, same date window	"Everything else going on in this window, for contention/context."

This is intentionally the opposite of the earlier, narrower behavior ("this WO's own tree") — the change is explicit and permanent, not a bug being reverted.

12. Performance — Page Background Task Pagination

Added 2026-09-03, on explicit user request. As Section 4's cross-Work-Order scope (§11) made the add-in query every other Work Order's Day Planning demand company-wide for the visible window, real companies with a large Day Planning volume for that period made JSON generation (AL) and DHTMLX ingest (JS parse + tree/model build + render) slow enough to notice on first paint. The fix ports this repository's existing **Page Background Task** pattern — already used by `request_assignment` (codeunit 50721), `projectschedule` (codeunit 50720), and `ganttdemo2` (codeunit 50713) — into CPO, following the exact same shape.

12.1 The technique (Business Central Page Background Task)

API	Role
<code>CurrPage.EnqueueBackgroundTask(TaskId, Codeunit, Parameters, TimeoutMs, ErrorLevel)</code>	Starts a child session running the given codeunit's <code>OnRun</code> trigger asynchronously, off the interactive request path. Returns immediately.
<code>Page.GetBackgroundParameters(): Dictionary of [Text, Text]</code>	Read inside the child codeunit's <code>OnRun</code> to retrieve the parameters passed at enqueue time.
<code>Page.SetBackgroundTaskResult(Dictionary of [Text, Text])</code>	Called at the end of the child codeunit's <code>OnRun</code> to hand results back to the parent page.
<code>trigger OnPageBackgroundTaskCompleted(TaskId, Results)</code>	Fires on the parent page when the child session finishes. Cannot safely call the control add-in from here (confirmed live, same finding as codeunits 50713/50720/50721) — must stash the result into a plain AL variable instead.
<code>trigger OnPageBackgroundTaskError(TaskId, ErrorCode, ErrorText, ErrorCallStack, var IsHandled)</code>	Fires on failure/timeout. Surfaced via a <code>Notification</code> , not a blocking error — the first page already rendered successfully.

Why results can't be pushed to the control add-in directly from `OnPageBackgroundTaskCompleted` : confirmed live across four independent add-ins in this repository (Gantt, Task Scheduler, Request/Assignment Board, and now CPO) — BC Server rejects a control add-in callback issued directly from that trigger. The workaround, used identically everywhere: stash the result into a plain page variable, then let a **JS-initiated poll** (a normal synchronous trigger call, safe to answer with a control add-in callback) pick it up and deliver it.

12.2 CPO's architecture (mirrors `request_assignment`'s request/assignment board exactly)

```
page 50722 RefreshData()
→ CPO_BuildPlanningDataJson_Paged(WorkOrderNo, DaysToShow, MaxOtherLines=50, out RemainingGroupKeys)
→ CurrPage.DhxCpo.SetPlanningData(json) // first paint: fast
→ EnqueueOtherWorkOrderDataBackgroundTask(RemainingGroupKeys)
  → CurrPage.EnqueueBackgroundTask(.., Codeunit::"CPO BG Other WO Data", params, 30000, ..)
  → CurrPage.DhxCpo.NotifyOtherWorkOrderDataTaskPending() // JS starts polling

  (async, off the interactive request path)
  codeunit "CPO BG Other WO Data".OnRun()
    → CPO_BuildOtherWorkOrderLinesJson_ForKeys(WorkOrderNo, StartDate, EndDate, RemainingGroupKeys)
    → Page.SetBackgroundTaskResult({ otherWorkOrderDataJson: json })

← trigger OnPageBackgroundTaskCompleted(TaskId, Results)
  stash Results into PendingOtherWorkOrderDataJson (plain var - no control add-in call here)

← trigger OnPollOtherWorkOrderDataResult() // JS-initiated, polled every 500ms
  CurrPage.DhxCpo.AppendOtherWorkOrderData(PendingOtherWorkOrderDataJson)
  CurrPage.DhxCpo.StopOtherWorkOrderDataPolling()
```

12.3 The atomic pagination unit — whole Skill+Job No.+Job Task No. groups, never split

Exactly like `request_assignment`'s `sequenceKey` pagination, CPO never splits a rendering unit across the synchronous first page and the background remainder. The unit here is a whole **group** — Skill+Job No.+Job Task No., the same combination Section 4's tree groups by (`CPO_BuildGroupsArray`). A two-pass split (mirroring `ReqAssign_BuildDayTaskLinesJson_Paged`'s own shape) makes this both correct and cheap:

Pass	Cost	Scope	Produces
<code>CPO_ScanOtherWorkOrderGroups</code> (Pass 3a)	Cheap — key fields only (Job No./Job Task No./Skill/Description), no per-line JSON	ALWAYS full — every group in the whole visible window, never paginated	Complete <code>skills[] / resources[] / externalFree[] / groups[]</code> (the tree <u>skeleton</u>) + the group order/line-count needed to decide the page cutoff
<code>CPO_BuildOtherWorkOrderLinesForGroups</code> (Pass 3b)	Expensive — Job/Job Task description lookups, time-to-decimal conversions, full per-line JSON (<code>CPO_BuildDayPlanningLineObj</code>)	Paginated — only whichever whole groups are in the caller's <code>WantedGroupKeys</code>	<code>dayPlanningLines[]</code> entries — the expensive part, and the only part actually deferred

Practical effect: Section 4's tree skeleton (every Skill/Job/Task node) renders complete on first paint, even over a huge dataset — a few hundred distinct groups costs nothing to pre-scan. Only the Sequence-level leaf chips for groups past the first 50 backfill progressively once the background task delivers them, typically

within the same second or two on this demo dataset (confirmed live: 1,388 distinct groups across 8 skills, 5,042 total Day Planning lines merged successfully with zero console errors).

12.4 Why Section 1/2/3 are treated differently

Section	Paginated?	Reason
Section 2 (inspected WO's own scheduler)	No — Pass 1/2 unchanged	Bounded to one Work Order's own lines — never the actual bottleneck the user identified.
Section 1 (stats) / Section 3 (capacity bars)	Indirectly — provisional until backfill completes	Both are <i>computed</i> from the full company-wide <code>dayPlanningLines[]</code> (the max-flow engine's <code>capParts()</code> /baseline). Their shortage/coverage numbers are correct for whatever data has arrived so far, and self-correct once the background remainder is appended.
Section 4 (tree)	Yes — the actual target	Skeleton always complete; chip-level detail backfills per §12.3.

12.5 The JS-side append — why it's more than a tree rebuild

`request_assignment`'s own `AppendDayTaskLines` only needs to `.push()` the new rows and rebuild a display tree — its board doesn't derive anything from the rows beyond how to group them. CPO is different: its shortage/coverage engine (§9) and Section 3's capacity bars are *computed* from the full `dayPlanningLines[]` set, so `appendOtherWorkOrderData(rawLines)` in `capacityPlanningOverview.js` must:

1. Push the new lines into `this.db.dayPlanningLines` (no `groups[]` merge needed — that structure was already complete from the first payload, per §12.3).
2. Invalidate every derived cache: `this._evalWOCache`, `this._currentPositionShortageArr`, `this._currentPositionSkillShortageArr`, and rebuild `this._baseRequests` / `this._baselineWithoutWO` — the exact same caches `applyPlanningData()` itself resets on a full load.
3. Re-render Sections 1, 3, and 4 (`renderWoSummaryScheduler` / `renderCapacityBars` / `renderCentralTree`) and re-bind the shared scrollbar. Section 2 is not re-rendered — its own source data was never paginated, so nothing it shows can change.

The polling mechanism itself (`wrapper.js`'s `NotifyOtherWorkOrderDataTaskPending` / `StopOtherWorkOrderDataPolling`) is a byte-for-byte port of `request_assignment/wrapper.js`'s identical functions — a bounded 500 ms/60-attempt (30s ceiling) `setInterval` loop calling `InvokeExtensibilityMethod("OnPollOtherWorkOrderDataResult", [])`, stopped early once AL actually delivers a result.

12.6 New/changed objects

Object	Change
codeunit 50722 "CPO BG Other WO Data"	New. Background task target — <code>OnRun()</code> reads parameters (<code>WorkOrderNo</code> , <code>StartDate</code> , <code>EndDate</code> , <code>RemainingGroupKeys</code>), calls <code>CPO_BuildOtherWorkOrderLinesJson_ForKeys</code> , returns the result. Read-only, its own session/transaction — modeled directly on codeunit 50721.
codeunit 50604: <code>CPO_BuildPlanningDataJson_Paged</code>	New. Paged variant of <code>CPO_BuildPlanningDataJson</code> — see §12.3's two-pass split.
codeunit 50604: <code>CPO_ScanOtherWorkOrderGroups</code>	New. Pass 3a — the cheap, always-full pre-scan.
codeunit 50604: <code>CPO_BuildOtherWorkOrderLinesForGroups</code>	New. Pass 3b — shared by both the paged builder's first page and the background task's remainder.
codeunit 50604: <code>CPO_BuildOtherWorkOrderLinesJson_ForKeys</code>	New. Background-task companion — public entry point called from codeunit 50722.
page 50722	<code>RefreshData()</code> now calls the paged builder + <code>EnqueueOtherWorkOrderDataBackgroundTask</code> ; new <code>OnPageBackgroundTaskCompleted</code> / <code>OnPageBackgroundTaskError</code> triggers; new <code>OnPollOtherWorkOrderDataResult</code> trigger inside the <code>usercontrol</code> block; new vars <code>OtherWorkOrderDataTaskId</code> / <code>PendingOtherWorkOrderDataJson</code> / <code>PendingOtherWorkOrderDataAvailable</code> .
Controladdin	New procedures <code>AppendOtherWorkOrderData</code> / <code>NotifyOtherWorkOrderDataTaskPending</code> / <code>StopOtherWorkOrderDataPolling</code> ; new event <code>OnPollOtherWorkOrderDataResult</code> .
wrapper.js	New poll timer + <code>window.AppendOtherWorkOrderData</code> , delegating to the component instance.
capacityPlanningOverview.js	New <code>appendOtherWorkOrderData(rawLines)</code> method — see §12.5.

13. Known Pitfalls, Bugs Found & Fixes Applied (session log)

12.1 Unterminated CSS comment silently disabled the top bar

`style.css` had a comment inside `#cpo-root {}` that opened `(/*`) but never closed before that block's own `}` — the first real `*/` the parser then found was ~30 lines later, silently swallowing `.cpo-top-bar`'s and `.cpo-top-title`'s entire rule bodies (`display:flex`, the yellow background, font styling — all of it) as dead comment text. Symptom: the title and "Days to show" input stacked on two plain, unstyled rows instead of one styled row. **Fix:** add the missing `*/` immediately after the intended one-paragraph comment.

Lesson: an unterminated CSS comment doesn't error — it silently consumes everything until the next literal `*/` anywhere later in the file, which can span multiple unrelated, syntactically-valid-looking rules. Always grep for balanced `/* */` pairs when a rule that "should" apply visibly doesn't.

12.2 `RunModal()` vs `Run()` — native BC chrome

Opening the page via `RunModal()` always renders BC's own dialog chrome (bold title bar + Manage/Actions/Fewer options command bar + Close button) regardless of the page's own `Caption` or field content — that chrome is a property of modal dialogs generally, not of this specific page. Switching to `Run()` (plain top-level navigation, the same pattern `page 50710 "DHX Request Assignment Board"` already used from a Role Center tile) replaces it with just a back arrow, matching the reference prototype's own minimal chrome. The page's own live actions (if any) still render a slim command bar in this mode — removing the `ResetPositionAct` action entirely eliminated that too.

12.3 "Work Order No." field vs Job No./Job Task No. scoping

See §6/§8/§11. The native Workorder Card's own "Day Planning Sequence" part filters Day Planning by Job No./Job Task No. (via the Work Order's "Project No." / "Project Task No."), never by the "Work Order No." field. Some genuine Day Planning Sequences (e.g. manually added via that native part's "New sequence" button) can have that field blank. Any AL query that scopes "this Work Order's own data" by the "Work Order No." field alone will silently miss such rows — and, symmetrically, any "NOT this Work Order" exclusion keyed the same way will wrongly include them as if they belonged to someone else.

12.4 Flexbox default shrink crushed the shared scrollbar

`#cpo-root` is a fixed-height `display:flex; flex-direction:column` container. `#cpo-shared-scroll` (declared `height:16px`) had no `flex-shrink` override, so whenever the other sections' combined natural height exceeded `#cpo-root`'s own bounded height, the flex column's default `flex-shrink:1` proportionally squashed **every** child — and a tiny 16px sibling shrinks to near-nothing far faster than a 400px one. Measured live: rendered height collapsed to 1px. **Fix:** `flex: 0 0 16px;` (no shrink, no grow, fixed 16px basis).

12.5 DHTMLX auto-fits row height to its container — scrollbars never appear

Even with `dy / folder_dy` fixed and `section_autoheight:false`, DHTMLX Scheduler reads its own init-target element's `clientHeight` at render time and fits all row content to exactly that height (shrinking rows) rather than rendering at the literal configured height and overflowing. The wrapping div's `overflow-y:auto` therefore never has anything genuinely taller than itself to scroll. **Fix (applied to both Section 2 and Section 4):** split the single host div into two nested divs — DHTMLX initializes into an inner div left height-unconstrained (so it grows to its true natural `rowCount × dy` height), while a new outer wrapper div carries the MIN/MAX-clamped height and `overflow-y:auto` that actually does the visual clipping/scrolling. Horizontal scroll-sync (`getScrollOwner()`), which looks up DHTMLX's own internal `.dhx_timeline_data_wrapper` is unaffected since it still queries by the same inner element's id.

12.6 Concurrent publish/build collisions

Running multiple `al_publish` operations from parallel sessions/subagents against the same sandbox environment can produce a spurious `CompilationFailed` even when `al_getdiagnostics` shows zero real errors — a file-lock/build collision, not a code defect. Symptom looked identical to a browser-caching issue at first; resolved by waiting for the other publish to finish and retrying.

14. Appendix — File Reference

Path (relative to repo root)	Contents
src\dhx\capacity_planning_overview\DHXCapacityPlanningOverviewAddin.ControlAddin.al	Controladdin declaration
src\dhx\capacity_planning_overview\page_50722_CapacityPlanningOverview.al	Host page 50722
src\dhx\capacity_planning_overview\startupScript.js	BOOT() call
src\dhx\capacity_planning_overview\wrapper.js	BOOT/binding shell
src\dhx\capacity_planning_overview\capacityPlanningOverview.js	The single JS component (~1500 lines) — all 4 sections + shortage engine
src\dhx\capacity_planning_overview\style.css	All layout/heat-map/chip/scrollbar CSS
src\codeunit\codeunit_50604_DHXDataHandler.al	Shared data-handler codeunit — CPO_ region builds the payload (incl. the paged/background-task builders, §12)
src\dhx\capacity_planning_overview\codeunit_50722_CPOBG0therWorkOrderData.al	Page Background Task target — §12
src\page\Pag50662.WorkorderCard.al	Launching action CapacityPlanningOverviewAct
src\dhx\dhtmlxscheduler.js / .css	Shared vendor Scheduler library (root-level, reused by every DHX add-in)
src\dhx\suite.js / .css	Shared vendor Suite library (Layout/Window/Grid — declared but not currently used by this add-in's own modal, which is stubbed)
C:\Users\Ahmad\OneDrive\x\PROJECT\KLAAS\CPO_v131\DHTMLXtempv112-app.js	Reference prototype this add-in is ported from (external to this repo)

End of document.